UADER - FCyT | Ingeniería de Software II

Guion Paso a Paso para la Defensa Oral: TPFI

Proyecto: TPFI-PROJECT | **Autores:** Camila Viñeg, Tadeo Rochas, Abril Poggio | **Versión:** 0.5.0

0. Disposición de la Pantalla (3 Terminales de PowerShell)

Antes de que te llamen, acomoda tus ventanas en la pantalla de la siguiente manera y verifica tener activo el entorno `(.venv)`:

```
+----------------------------------+----------------------------------+
|            TERMINAL 1            |            TERMINAL 2            |
|     (Servidor TCP Multihilo)     |       (ObserverClient Push)      |
+----------------------------------+----------------------------------+
|                           TERMINAL 3                                 |
|            (SingletonClient - Peticiones Transaccionales)            |
+----------------------------------------------------------------------+
```

1. Introducción Inicial (30 segundos)

> **QUÉ DECIR AL TRIBUNAL:**
> *"Buenas tardes profesores. Nuestro proyecto es una aplicación distribuida cliente-servidor desarrollada sobre TCP y conectada a AWS DynamoDB.*
> *El sistema implementa cuatro patrones de diseño GoF:*
> *1. **Singleton:** para gestionar el acceso físico único a las tablas `CorporateData` y `CorporateLog`.*
> *2. **Proxy:** para interceptar, validar datos mínimos y proteger las claves primarias antes de acceder a la base.*
> *3. **Observer:** para notificar en tiempo real a clientes suscriptos cada vez que un registro cambia.*
> *4. **Iterator:** para recorrer y paginar transparentemente DynamoDB sin truncamientos ni saturación de memoria.*
> *A continuación, realizaremos la demostración funcional en vivo con 3 servicios concurrentes."*

2. Puesta en Marcha de Servicios (1 minuto)

Paso 2.1: Iniciar el Servidor de Aplicaciones (Terminal 1)

```
python -m tpfi.singletonproxyobserver -p 8080 -v
```

> **QUÉ DECIR:**
> *"Iniciamos en la Terminal 1 el servidor multihilo en el puerto 8080. Como pueden observar en los logs, los repositorios se inicializan bajo el patrón Singleton: una única instancia para todo el proceso. Cada cliente entrante será atendido en un hilo independiente para garantizar concurrencia."*

Paso 2.2: Conectar el Observador Reactivo (Terminal 2)

```
python -m tpfi.observerclient -s 127.0.0.1 -p 8080 -v
```

> **QUÉ DECIR:**
> *"En la Terminal 2 conectamos el `ObserverClient`. El observador envía su UUID de CPU solicitando la suscripción. El servidor confirma con `status: ok` y deja el socket TCP abierto en estado de escucha activa para recibir notificaciones push."*

3. Batería de Pruebas en Vivo (Terminal 3: SingletonClient)

Ejecuta los siguientes casos sucesivamente desde la **Terminal 3**:

Caso 3.1: Consulta puntual por clave primaria (Acción 'get')

```
python -m tpfi.singletonclient -s 127.0.0.1 -p 8080 -i "samples/input_get.json" -v
```

> **QUÉ SEÑALAR Y DECIR:**
> ***1. Señalar Terminal 3:*** *"Enviamos la acción `get` solicitando la sede `UADER-FCyT-IS2`. El servidor nos devuelve los datos completos con `status: ok`."*
> ***2. Señalar Terminal 1 (Servidor):*** *"Fíjense en los logs del servidor: antes de resolver la lectura, se registró obligatoriamente la pista de auditoría en la tabla `CorporateLog` con marca temporal UTC y la telemetría de CPU del cliente."*

Caso 3.2: Alta del registro demo y notificación en cascada (Acción 'set') ★ MOMENTO CLAVE

```
python -m tpfi.singletonclient -s 127.0.0.1 -p 8080 -i "samples/input_set.json" -v
```

> **QUÉ SEÑALAR Y DECIR:**
> ***1. Señalar Terminal 3:*** *"Enviamos un `set` con el registro descartable `TPFI-DEMO-2026` y todos los campos del tuple. El Proxy valida los datos y el repositorio crea el registro, o lo actualiza si ya existe, mediante una operación atómica en DynamoDB."*
> ***2. Señalar inmediatamente la Terminal 2 (Observador):***
> ***"¡Miren la Terminal 2! Sin hacer ninguna recarga ni polling, el observador acaba de recibir la notificación push en cascada en tiempo real con el registro actualizado."***

Caso 3.3: Listado completo y paginación transparente (Acción 'list')

```
python -m tpfi.singletonclient -s 127.0.0.1 -p 8080 -i "samples/input_list.json" -v
```

> **QUÉ DECIR:**
> *"Pedimos el listado completo. Aquí entra en juego el **patrón Iterator**: DynamoDB limita las lecturas de `scan` a 1 MB por llamada. Nuestro `RecordIterator` recorre página por página siguiendo `LastEvaluatedKey` de forma perezosa (lazy evaluation), trayendo toda la tabla sin truncamientos y sin saturar la memoria RAM del servidor."*

Caso 3.4: Validación y protección del Proxy ('set' inválido)

```
python -m tpfi.singletonclient -s 127.0.0.1 -p 8080 -i "samples/input_invalido.json" -v
```

> **QUÉ DECIR:**
> *"Enviamos una petición donde el cliente intenta modificar el campo reservado `id` (clave primaria). Como ven, **el Proxy interceptó el requerimiento y lo rechazó con `status: error`**. La petición ni siquiera llegó a la base física, protegiendo la integridad de la base de datos."*

4. Cierre y Verificación Automatizada (1 minuto)

Paso 4.1: Suite de Pruebas Automatizadas

```
pytest
```

> **QUÉ DECIR:**
> *"El proyecto cuenta con **195 pruebas automatizadas** que cubren pruebas unitarias con dobles de prueba, la suite formal de **pruebas de aceptación** (`test_acceptance.py`) con los 5 escenarios obligatorios de la consigna, y pruebas de propiedades con **Hypothesis** (`test_hypothesis.py`) sometiendo al protocolo a fuzzing. Alcanzamos un **95.03% de cobertura total**."*

Paso 4.2: Chequeo de Linters y Estilo

```
ruff check . ; black --check .
```

> **QUÉ DECIR:**
> *"El código aprueba al 100% el análisis estático de linters (Ruff), formato estándar PEP 8 (Black), tipado estricto (Mypy) y seguridad (Bandit)."*

5. Respuestas Rápidas para Preguntas del Tribunal

- **¿Por qué `session_id` lo crea el servidor y no el cliente?**
  Respuesta: "Por seguridad y no repudio. Si el cliente lo eligiera, podría sobrescribir auditorías anteriores o falsear identidades en `CorporateLog`."
- **¿Qué pasa si se cae el observador?**
  Respuesta: "El servidor detecta el socket roto, hace una poda automática (`unsubscribe`) para no dejar descriptores zombis, y el cliente tiene un bucle que reintenta reconectarse cada 30 segundos."
- **¿Por qué usan delimitador `\n` en los sockets?**
  Respuesta: "Porque TCP es un flujo continuo de bytes (stream). Sin encuadre por delimitador, los mensajes podrían mezclarse o fragmentarse. `JsonSocket` bufferiza y entrega mensajes JSON completos."
- **¿Qué pasa si falla Internet en el aula?**
  Respuesta: "Tenemos preparado el script `python demo_runner.py` que emula el ciclo completo contra repositorios en memoria de forma 100% offline."